MODULE 1

INTRODUCTION TO DATA SCIENCE

Definitions • Skills • Tools • Data Types • Collections • Preprocessing • Analysis & Analytics

Study Notes

This module introduces the basic concepts of Data Science, the skills and tools required by a data scientist, common data types and collections, data preprocessing, and the major types of data analysis.

1. Definition of Data Science

Data Science is an interdisciplinary field that uses statistics, mathematics, programming, data analysis, machine learning, and domain knowledge to extract useful information and insights from data.

Key idea: Data Science converts raw data into meaningful insights that can support decisions, predictions, and problem solving.

Typical Data Science workflow:

- Collect data from databases, files, sensors, websites, applications, or APIs.
- Clean and preprocess the collected data.
- Explore and analyze the data.
- Build statistical or machine-learning models when required.
- Evaluate the results.
- Communicate insights using reports, dashboards, or visualizations.

2. Skills Required for Data Science

A data scientist needs a combination of technical, analytical, and communication skills.

Technical skills

- Programming – Python, R, Java, or other languages.
- Statistics and probability – distributions, hypothesis testing, correlation, regression, etc.
- Mathematics – linear algebra, calculus, and optimization basics.
- Data handling – cleaning, transformation, and database operations.
- Machine Learning – supervised and unsupervised learning concepts.
- Data visualization – charts, dashboards, and visual storytelling.

Non-technical skills

- Problem-solving and critical thinking.
- Domain knowledge.
- Communication and presentation.
- Business understanding and decision-making.

3. Tools for Data Science

Data scientists use different tools for programming, data storage, analysis, visualization, and machine learning.

Category	Examples	Purpose
Programming	Python, R, SQL	Programming, querying and data manipulation
Development	Jupyter Notebook, VS Code	Writing and running analysis/code
Python libraries	NumPy, Pandas, Matplotlib, Seaborn	Numerical work, data handling and visualization
Machine Learning	Scikit-learn, TensorFlow, PyTorch	Model development
Databases	MySQL, PostgreSQL, MongoDB	Data storage and retrieval
Visualization	Power BI, Tableau	Dashboards and business reporting

4. Data Types

A data type describes the kind of value stored in a variable or dataset column.

Common data types

- Numerical: numbers used for measurements or quantities. Example: age, salary, height.
- Integer: whole numbers. Example: 10, 25, 100.
- Float: decimal numbers. Example: 72.5, 3.14.
- Categorical: values representing groups or categories. Example: Department = HR, Sales, IT.
- Binary: two possible values, such as Yes/No or 0/1.
- Ordinal: categories with an order. Example: Low, Medium, High.
- Text/String: words or sentences. Example: customer comments.
- Date/Time: dates and timestamps. Example: 2026-10-06.

Important: Correctly identifying data types is necessary before analysis because different types require different operations.

5. Data Collections

Data collections are structures used to store and organize multiple values. In Python, the main built-in collections are list, tuple, set, and dictionary.

Collection	Characteristics	Example
List	Ordered, changeable, allows duplicates	[10, 20, 20, 30]
Tuple	Ordered, generally immutable, allows duplicates	(10, 20, 30)
Set	Unordered collection of unique values	{10, 20, 30}
Dictionary	Key-value pairs	{'name': 'Ravi', 'age': 20}

6. Data Preprocessing

Data preprocessing is the process of preparing raw data so that it becomes suitable for analysis and machine-learning tasks.

Main preprocessing steps

- Data cleaning – handle missing values, duplicates, incorrect values, and inconsistent formats.

- Data integration – combine data from multiple sources.
- Data transformation – change data into a suitable form.
- Encoding – convert categorical values into numerical representations when required.
- Feature scaling – bring numerical features to comparable scales using methods such as normalization or standardization.
- Outlier handling – identify and appropriately treat unusual observations.
- Feature selection – retain relevant variables and remove unnecessary ones.

Example: A salary column containing ₹50,000, "50000", and missing values needs cleaning and conversion before reliable numerical analysis.

7. Data Analysis and Data Analytics

Data analysis is the process of inspecting, cleaning, transforming, and interpreting data to discover useful information. Data analytics is the broader practice of using data, analytical methods, tools, and models to answer questions and support decisions.

Data Analysis

- Focuses on examining data and finding patterns, relationships, trends, and abnormalities.

Data Analytics

- Uses analysis along with statistical, computational, and business techniques to generate actionable insights.

Example: An analyst may analyze monthly sales data to discover that sales are falling. Analytics can go further by identifying likely causes and recommending actions.

8. Descriptive Analysis

Descriptive analysis answers: “What happened?” It summarizes historical data using numbers, tables, and visualizations.

- Mean, median, mode, minimum, maximum, and standard deviation.
- Frequency tables and percentages.
- Bar charts, line charts, histograms, and dashboards.

Example: “The company sold 12,000 units in September, which was 8% higher than August.”

9. Diagnostic Analysis

Diagnostic analysis answers: “Why did it happen?” It investigates relationships, patterns, and possible causes behind an observed result.

- Compare groups or time periods.
- Drill down into detailed data.
- Examine correlations and relationships.
- Identify factors associated with an outcome.

Example: If September sales increased, diagnostic analysis may determine that a festival promotion and discount campaign contributed to the increase.

10. Predictive Analysis

Predictive analysis answers: “What is likely to happen?” It uses historical data, statistics, and machine-learning models to estimate future or unknown outcomes.

- Regression can predict numerical values such as sales or price.
- Classification can predict categories such as churn/not churn.
- Time-series methods can forecast future values.

Example: A company predicts which customers are likely to stop using its service next month.

11. Prescriptive Analysis

Prescriptive analysis answers: “What should we do?” It uses analytical results, constraints, and sometimes optimization or simulation to recommend actions.

- Suggests possible decisions or actions.
- Can compare alternative strategies.
- May use optimization, simulation, rules, or predictive-model outputs.

Example: If demand is predicted to increase, prescriptive analysis may recommend increasing inventory by a suitable amount.

12. Exploratory Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of examining a dataset to understand its structure, distributions, relationships, missing values, outliers, and important patterns before formal modeling.

Common EDA activities

- Inspect rows, columns, data types, and summary statistics.
- Check missing values and duplicates.
- Study distributions using histograms and density plots.
- Study relationships using scatter plots, correlation matrices, and grouped summaries.
- Identify outliers and unusual observations.
- Generate hypotheses for further analysis.

Example: Before building a salary-prediction model, EDA can reveal missing salaries, unusual salary values, department differences, and relationships with experience.

13. Mechanistic Analysis

Mechanistic analysis focuses on understanding the underlying mechanism or physical/process relationships that explain how inputs produce an outcome. It is more concerned with the structure of the process than simply finding statistical association.

- Attempts to explain how a system works.
- Uses domain knowledge, mathematical relationships, or physical/process rules.
- Can be useful when cause-and-effect mechanisms are known or can be modeled.

Example: In an engineering system, mechanistic analysis may model how temperature, pressure, and flow rate interact to determine system behavior.

14. Types of Analysis – Quick Comparison

Type	Main Question	Purpose
Descriptive	What happened?	Summarize past data

Type	Main Question	Purpose
Diagnostic	Why did it happen?	Find causes and relationships
Predictive	What is likely to happen?	Forecast future/unknown outcomes
Prescriptive	What should we do?	Recommend actions
Exploratory	What patterns exist?	Discover patterns and generate hypotheses
Mechanistic	How does the system work?	Model underlying mechanisms

15. Important Exam Points

- Data Science: combines programming, statistics, mathematics, domain knowledge, and analytical methods to extract useful insights from data.
- Data preprocessing: prepares raw data by cleaning, transforming, integrating, encoding, scaling, and handling outliers.
- Descriptive: What happened?
- Diagnostic: Why did it happen?
- Predictive: What is likely to happen?
- Prescriptive: What should we do?
- Exploratory: Discover patterns and relationships in data.
- Mechanistic: Understand the underlying mechanism of a system.

MODULE 2

DATA EXTRACTION

Feature Selection • Filters • Wrappers • Entropy • Decision Trees • Random Forest • Dimensionality Reduction • SVD • PCA

Study Notes

This module introduces techniques used to select useful features, reduce unnecessary dimensions, and build tree-based machine learning models. The notes are written in an exam-friendly format with definitions, working steps, advantages, limitations, and examples.

1. Feature Selection

Feature selection is the process of selecting the most relevant input variables (features) from a dataset while removing irrelevant or redundant features.

Why feature selection is needed

- Reduces the number of input variables.
- Removes irrelevant or noisy features.
- Can improve model accuracy and generalization.
- Reduces training time and memory usage.
- Makes the model easier to understand and interpret.

Example: For predicting student performance, features such as attendance, previous marks, and study hours may be useful, while an unrelated ID number usually provides no useful information.

Main approaches: Filter methods, Wrapper methods, and Embedded methods.

2. User Retention

User retention refers to the ability of a product, application, or service to keep users active over a period of time. In data analysis, retention is often studied using historical user activity data.

Retention rate

A simple retention-rate calculation is:

> **Retention Rate = (Number of users retained during a period / Number of users at the beginning of the period) × 100**

- Example: If 800 users start a month and 600 are still active at the end, retention rate = 75%.

Features useful for retention analysis: login frequency, session duration, purchases, subscription status, last activity date, and number of interactions.

Relation to feature selection: selecting the strongest behavioral features can help a model identify users who are likely to remain active or leave.

3. Filter Methods

Filter methods select features using statistical properties of the data before a machine-learning model is trained. They are generally fast because they are model-independent.

Common filter techniques

- Correlation – identifies strongly related variables.
- Chi-square test – commonly used for categorical features and categorical targets.
- ANOVA / F-test – can compare differences between groups.
- Mutual information – measures how much information one variable provides about another.
- Variance threshold – removes features with very low variance.

Advantages: fast, simple, scalable, and independent of the final model.

Limitation: a filter method may select features that look individually useful but are not optimal when combined with other features.

4. Wrapper Methods

Wrapper methods evaluate different subsets of features by actually training and evaluating a machine-learning model. The model's performance is used to decide which feature subset is better.

Common wrapper techniques

- Forward Selection – starts with no features and adds useful features one by one.
- Backward Elimination – starts with all features and removes less useful features.
- Recursive Feature Elimination (RFE) – repeatedly removes the least important features.

Basic process: Choose a feature subset → train the model → evaluate performance → change the subset → repeat → select the best subset.

Advantages: considers feature interactions and is closely related to the chosen model.

Limitation: computationally expensive because many models may need to be trained.

5. Entropy

Entropy measures the impurity or uncertainty in a dataset. It is commonly used in decision trees to decide which feature should be used for splitting.

For a classification problem, entropy is:

$$\textbf{Entropy}(S) = -\sum p_i \log_2(p_i)$$

- p_i = proportion of samples belonging to class i.
- Entropy = 0 when all samples belong to one class.
- Higher entropy means greater uncertainty or impurity.

Example: If a node contains only Positive examples, its entropy is 0. If Positive and Negative examples are equally divided, entropy is 1.

6. Decision Tree Algorithm

A decision tree is a supervised learning algorithm that makes decisions by repeatedly splitting data according to feature values. It can be used for classification and regression.

Important parts

- Root node – first and topmost decision.
- Internal node – represents a decision or test on a feature.
- Branch – represents an outcome of a decision.
- Leaf node – represents the final prediction.

Working steps

- Start with the complete training dataset.
- Calculate a splitting criterion such as information gain or Gini impurity.
- Choose the best feature/split.
- Split the data into child nodes.
- Repeat the process for the child nodes.
- Stop when a stopping condition is reached, such as maximum depth or sufficient purity.

Information Gain:

Information Gain = Entropy(parent) − Weighted Entropy(children)

Advantages: easy to interpret, handles nonlinear relationships, and generally requires little feature scaling.

Limitations: can overfit, especially when the tree becomes very deep; pruning or depth restrictions can help.

7. Random Forest

Random Forest is an ensemble learning algorithm that combines many decision trees to produce a more robust prediction.

Working principle

- Create multiple training samples using bootstrap sampling.
- Build a decision tree for each sample.
- At each split, consider a random subset of features.
- For classification, combine tree predictions using majority voting.
- For regression, commonly average the predictions.

Why it works: individual trees can have high variance. Combining many diverse trees generally improves stability and reduces overfitting compared with a single unrestricted tree.

Advantages: strong performance, handles many features, less sensitive to noise than a single tree, and can estimate feature importance.

Limitation: less interpretable than one decision tree and can require more computation and memory.

8. Dimensionality Problems

Dimensionality refers to the number of features or variables in a dataset. A dataset with many features is called high-dimensional.

Problems caused by high dimensionality

- Curse of dimensionality – data become sparse as the number of dimensions increases.

- Higher computational and storage cost.
- More irrelevant and redundant information.
- Greater risk of overfitting.
- Difficulty in visualizing and interpreting the data.

Solution: dimensionality can be reduced through feature selection or feature extraction techniques such as PCA and SVD.

9. Singular Value Decomposition (SVD)

Singular Value Decomposition decomposes a matrix into three matrices. It is widely used for dimensionality reduction, noise reduction, data compression, and recommender systems.

For a matrix A, SVD is represented as:

$$\mathbf{A = U\Sigma V^T}$$

- U – left singular vectors.
- Σ (Sigma) – diagonal matrix containing singular values.
- V^T – transpose of the right singular-vector matrix.

Basic idea: SVD represents the original matrix using directions that capture different amounts of information. By keeping only the largest singular values, a lower-dimensional approximation can be obtained.

Applications: image compression, text analysis, recommendation systems, noise reduction, and latent-feature extraction.

10. Principal Component Analysis (PCA)

PCA is a dimensionality-reduction technique that transforms correlated features into a smaller set of new variables called principal components.

Key idea

- The first principal component captures the maximum possible variance.
- The second principal component captures the maximum remaining variance while being orthogonal to the first.
- Further components capture progressively smaller amounts of variance.

Typical PCA steps

- Collect the numerical feature matrix.
- Standardize the features when their scales differ substantially.
- Calculate the covariance/correlation structure.
- Find principal directions using eigenvectors (or an SVD-based implementation).
- Rank components by explained variance.
- Keep the desired number of components and transform the data.

Advantages: reduces dimensions, removes some redundancy, supports visualization, and can speed up later modeling.

Limitations: transformed components can be difficult to interpret, scaling matters, and PCA is mainly designed to capture variance rather than directly optimize prediction accuracy.

11. Quick Comparison

The following table helps distinguish the major techniques in this module.

Technique	Main purpose	Model dependent?
Filter	Select features using statistical measures	No
Wrapper	Find feature subset using model performance	Yes
Decision Tree	Prediction using sequential feature-based splits	Yes – it is a model
Random Forest	Combine many decision trees	Yes – it is a model
SVD	Matrix decomposition / dimensionality reduction	No
PCA	Create fewer components that preserve variance	No

12. Important Exam Points

- Feature Selection: selecting relevant features and removing irrelevant or redundant ones.
- Filter: fast, model-independent feature selection based on statistical measures.
- Wrapper: evaluates feature subsets using model performance.
- Entropy: measures uncertainty/impurity in a dataset.
- Decision Tree: uses feature-based splits to reach a prediction at a leaf node.
- Random Forest: combines many decision trees using randomness and aggregation.
- Dimensionality: the number of features in a dataset; high dimensionality can cause sparsity, cost, and overfitting.
- SVD: decomposes a matrix as $A = U\Sigma V^T$.
- PCA: transforms features into principal components ordered by explained variance.

DATA SCIENCE

MODULE 3

FEATURE SELECTION AND DECISION TREE FOUNDATIONS

Study Notes • Definitions • Algorithms • Worked Examples • Exam Questions

Learning objectives

- Understand why feature selection is important in machine learning.
- Compare filter, wrapper, and embedded feature-selection methods.
- Explain entropy, information gain, and Gini impurity.
- Understand how a decision tree chooses splits and makes predictions.
- Describe overfitting, pruning, and practical evaluation.

Prepared as beginner-friendly classroom and exam revision notes.

1. Feature Selection

Definition: Feature selection is the process of choosing a useful subset of the available input variables (features) for building a machine-learning model. It removes irrelevant, redundant, or noisy features while retaining information useful for prediction.

Why is feature selection needed?

• **Improves generalization:** removing noise can help a model perform better on unseen data.

• **Reduces overfitting:** fewer unnecessary inputs can reduce the chance of learning accidental patterns.

• **Faster training:** fewer features can reduce computation and memory requirements.

• **Improves interpretability:** a smaller set of features is easier to explain.

• **Reduces data-collection cost:** in some applications, fewer measurements need to be collected.

Feature selection vs. feature extraction

Feature Selection	Feature Extraction
Selects some of the original features.	Creates new features by transforming or combining existing ones.
Original meaning is usually preserved.	New features may be harder to interpret.
Example: choose Age and Income from 20 columns.	Example: PCA transforms many variables into principal components.

Types of feature-selection methods

A. Filter methods

Filter methods score features using statistical properties of the data, without training a specific predictive model for every candidate subset. They are usually fast and model-independent.

• **Variance threshold:** removes features with almost no variation.

• **Correlation:** identifies features strongly related to the target; correlation between input features can also reveal redundancy.

• **Chi-square test:** commonly used to test association between categorical features and a categorical target; inputs must satisfy the method's requirements.

• **ANOVA F-test:** can compare the means of a numeric feature across target classes.

• **Mutual information:** measures how much knowing a feature reduces uncertainty about the target; it can detect nonlinear relationships.

Advantages: fast, simple, scalable. **Limitations:** may ignore interactions between features and the chosen model.

B. Wrapper methods

Wrapper methods evaluate feature subsets by training and validating a predictive model. They can capture how features work together, but may be computationally expensive.

• **Forward selection:** start with no features; add the feature that improves validation performance the most, repeating until a stopping rule is met.

• **Backward elimination:** start with all candidate features; remove the least useful feature step by step.

• **Recursive Feature Elimination (RFE):** fit a model, rank features by importance, remove the least important ones, and repeat.

Advantages: considers the model and feature combinations. **Limitations:** slower and more prone to overfitting if validation is not done carefully.

C. Embedded methods

Embedded methods perform feature selection as part of model training.

• **Lasso (L1 regularization):** can shrink some linear-model coefficients to exactly zero.

• **Decision trees:** select features for splits based on impurity reduction.

• **Random forests:** provide feature-importance measures aggregated across many trees; importance should be interpreted carefully, especially with correlated features.

Quick comparison: Filter = score features before model training; Wrapper = search subsets using model performance; Embedded = selection occurs during model training.

Important precautions

• Split the data into training and test sets before feature selection.

• Fit selection rules only on the training data, ideally inside each cross-validation fold, to avoid data leakage.

• Do not select features by looking at test-set performance.

• A feature's usefulness depends on the target, data quality, and model; importance does not automatically mean causation.

2. Decision Tree Foundations

Definition: A decision tree is a supervised-learning algorithm that makes predictions by repeatedly splitting data according to feature-based rules. It can be used for classification (predicting categories) and regression (predicting numeric values).

Main parts of a decision tree

• **Root node:** the first node, representing the full training dataset.

• **Internal/decision node:** a test on a feature, such as “Age ≤ 30?”.

• **Branch:** an outcome of a test, such as Yes/No.

• **Leaf/terminal node:** the final prediction, such as “Approved” or a numeric value.

How a decision tree works

1. Begin with the training examples at the root.
2. Evaluate possible splits on available features.
3. Choose a split that makes the child groups more pure or reduces prediction error.
4. Repeat the process within each child node.
5. Stop when a stopping condition is met; assign a prediction to each leaf.

Example rule path: If **Income = High** and **Credit History = Good**, then predict **Loan = Approve**. The tree is a collection of such decision rules.

3. Entropy and Information Gain

Entropy

Entropy measures the impurity or uncertainty of class labels in a node. For a classification node with class probabilities p■, p■, …, p■:

Entropy(S) = – Σ p■ log■(p■)

For binary classification, entropy is 0 when every example belongs to one class and reaches 1 when the two classes are equally represented. A common convention is to treat 0 × log■(0) as 0.

Worked entropy example

Suppose a node contains 10 students: 6 Pass and 4 Fail. Then p(Pass)=0.6 and p(Fail)=0.4.

Entropy = –[0.6 log■(0.6) + 0.4 log■(0.4)] ≈ 0.971 bits.

If all 10 students passed, entropy would be 0 because the node contains only one class.

Information Gain

Information Gain (IG) measures how much a split reduces entropy. It is the parent entropy minus the weighted average entropy of the child nodes.

IG(S, A) = Entropy(S) – Σ (|S■| / |S|) Entropy(S■)

The split with the highest information gain is preferred by the basic ID3 decision-tree algorithm.

Worked information-gain example

A dataset has 10 examples (6 Pass, 4 Fail), so parent entropy is approximately 0.971. A candidate split creates:

• Left child: 5 examples (all Pass), entropy = 0.

• Right child: 5 examples (1 Pass, 4 Fail), entropy ≈ 0.722.

Weighted child entropy = (5/10 × 0) + (5/10 × 0.722) = 0.361. Information Gain ≈ 0.971 – 0.361 = **0.610 bits**.

4. Gini Impurity and Split Criteria

Gini impurity is another measure of class mixing, commonly used by CART (Classification and Regression Trees).

Gini(S) = 1 – Σ p■²

For the 6 Pass / 4 Fail node: Gini = 1 – (0.6² + 0.4²) = 0.48. Gini is 0 for a pure node. A split is evaluated by the weighted impurity of its children; lower weighted Gini is better.

Criterion	Meaning	Typical use
Information Gain	Reduction in entropy after a split.	ID3
Gain Ratio	Information gain adjusted for the split's intrinsic information.	C4.5
Gini Impurity	Probability-based measure of class mixing.	CART classification trees
Variance / squared error reduction	Reduction in variation of numeric target values.	Regression trees

5. Common Decision-Tree Algorithms

ID3

ID3 builds a classification tree by choosing the feature with the highest information gain at each step. The original form is mainly associated with categorical attributes and can favor features with many distinct values.

C4.5

C4.5 extends ID3. It uses gain ratio, can handle continuous attributes by choosing thresholds, and includes pruning techniques.

CART

CART stands for Classification and Regression Trees. It typically creates binary splits, uses Gini impurity for classification and squared-error-based criteria for regression, and supports pruning.

Algorithm	Split criterion / characteristic
ID3	Information gain; classic version mainly categorical inputs.
C4.5	Gain ratio; supports continuous attributes and pruning.
CART	Binary splits; Gini for classification and error reduction for regression.

6. Overfitting, Stopping Rules, and Pruning

A very deep tree may memorize noise in the training data. It can achieve excellent training accuracy but perform poorly on new data; this is called overfitting.

Ways to control overfitting

• **Maximum depth:** limits how many levels the tree can grow.

• **Minimum samples per split/leaf:** prevents decisions based on very small groups.

• **Minimum impurity decrease:** requires a split to provide enough improvement.

• **Pre-pruning:** stop growth early using constraints.

• **Post-pruning:** grow a larger tree and then remove branches that do not improve generalization.

Underfitting occurs when a tree is too simple to capture useful patterns. Choose complexity using validation or cross-validation, then evaluate the final model once on held-out test data.

7. Advantages and Limitations of Decision Trees

Advantages	Limitations
Easy to visualize and explain.	Deep trees can overfit.
Can model nonlinear relationships and feature interactions.	Small data changes can produce a different tree.
Little feature scaling is usually needed.	Greedy split choices may not produce a globally optimal tree.
Can be used for classification and regression.	Some split criteria can be biased toward features with many possible values.

8. Feature Selection and Decision Trees Together

Decision trees can help with feature selection because they choose features that improve splits. Features never used in a fitted tree may be less useful for that particular tree, but this does not prove they are universally irrelevant. Correlated features can substitute for one another, and importance rankings may change across datasets.

• Use a training/validation process to compare feature subsets.

• Check whether selected features are stable across folds or repeated samples.

• Use domain knowledge as well as statistical scores.

• For high-dimensional data, a fast filter can narrow the candidate set before a more expensive wrapper or model-based approach.

9. Short Exam Questions and Answers

1. Define feature selection.

Feature selection is the process of selecting a subset of relevant input features for model building while removing irrelevant or redundant features.

2. Name the three major feature-selection approaches.

Filter methods, wrapper methods, and embedded methods.

3. What is a filter method?

A method that ranks or selects features using statistical properties without repeatedly training a predictive model for each subset.

4. What is a wrapper method?

A method that evaluates candidate feature subsets using the performance of a trained model.

5. What is an embedded method?

A method that performs feature selection as part of model training, such as Lasso or tree-based selection.

6. Define a decision tree.

A supervised-learning model that predicts by applying a sequence of feature-based tests arranged as a tree.

7. What is entropy?

Entropy is a measure of uncertainty or impurity in the class labels of a node.

8. Define information gain.

Information gain is the reduction in entropy caused by splitting a dataset on a feature.

9. What is Gini impurity?

Gini impurity measures class mixing and is calculated as 1 minus the sum of squared class probabilities.

10. What is pruning?

Pruning removes or limits branches to reduce overfitting and improve performance on unseen data.

11. Differentiate classification and regression trees.

Classification trees predict categorical labels; regression trees predict numeric values.

12. What is data leakage in feature selection?

Data leakage occurs when information from validation or test data influences feature selection or model training, leading to overly optimistic evaluation.

10. Long-Answer Practice

• Explain filter, wrapper, and embedded feature-selection methods with examples, advantages, and limitations.

• Describe the construction of a decision tree and explain root nodes, internal nodes, branches, and leaves.

• Define entropy and information gain. Show how information gain is used to select a split.

• Compare ID3, C4.5, and CART.

• Explain overfitting in decision trees and discuss pre-pruning and post-pruning.

• Discuss the advantages and limitations of decision trees in data science.

Quick Revision

Term	Remember
Feature selection	Choose a subset of original inputs.

Term	Remember
Filter / Wrapper / Embedded	Statistical score / model-subset search / selection during training.
Entropy	Higher when class labels are more mixed; zero for a pure node.
Information gain	Parent entropy minus weighted child entropy; higher is preferred by ID3.
Gini	1 – sum of squared class probabilities; lower impurity is better.
Pruning	Controls tree complexity to reduce overfitting.

DATA SCIENCE

MODULE 4

ENSEMBLE LEARNING

Study Notes • Core Concepts • Algorithms • Examples • Exam Questions

Learning objectives

• Explain ensemble learning and why combining models can improve predictions.

• Differentiate bagging, boosting, and stacking.

• Understand Random Forest, AdaBoost, Gradient Boosting, and XGBoost at a conceptual level.

• Describe voting, bootstrap sampling, out-of-bag evaluation, and feature importance.

• Recognize advantages, limitations, and common applications of ensemble methods.

Beginner-friendly classroom notes and examination revision material.

1. Introduction to Ensemble Learning

Definition: Ensemble learning is a machine-learning approach that combines predictions from multiple models, called base learners or weak learners, to produce a final prediction. The goal is often to improve accuracy, stability, and generalization compared with using one model alone.

Why combine models?

• **Reduce variance:** combining models can make predictions less sensitive to the particular training sample.

• **Reduce bias:** sequential methods such as boosting can improve models that underfit.

• **Improve robustness:** different models may make different errors, so combining them can help.

• **Increase predictive performance:** the ensemble can capture patterns that one model misses.

Important: an ensemble is not guaranteed to outperform every individual model. It works best when its base learners are reasonably useful and their errors are not all identical.

2. Main Types of Ensemble Learning

Method	Core idea	Training style	Common example
Bagging	Train models on different bootstrap samples and combine their outputs.	Usually parallel	Random Forest
Boosting	Train models sequentially, with later models focusing on errors or residuals.	Sequential	AdaBoost, Gradient Boosting, XGBoost
Stacking	Train a meta-model to combine predictions from different base models.	Base models then meta-model	Trees + linear model + meta-learner

3. Voting and Averaging

Hard voting

For classification, each model votes for a class. The class with the most votes becomes the ensemble prediction.

Example: Model 1 predicts Cat, Model 2 predicts Dog, and Model 3 predicts Cat. Hard voting predicts Cat because it receives two of the three votes.

Soft voting

Each classifier provides class probabilities. The probabilities are averaged (optionally with weights), and the class with the highest average probability is selected. Soft voting requires models whose probability outputs are meaningful and comparable.

Averaging for regression

For regression, predictions can be averaged. For example, if three models predict 20, 22, and 24, their simple average is (20 + 22 + 24) / 3 = 22.

Weighted voting or averaging gives more influence to models considered more reliable. Weights should be chosen using training/validation data, not the test set.

4. Bagging (Bootstrap Aggregating)

Definition: Bagging trains multiple versions of a model on different bootstrap samples of the training data and combines their predictions. A bootstrap sample is created by sampling training rows with replacement.

Steps in bagging

1. Draw multiple bootstrap samples from the training set.
2. Train one base model on each sample.

3. Collect each model's prediction for a new example.
4. Combine predictions using majority voting for classification or averaging for regression.

Why bagging helps

• It mainly reduces variance, especially for unstable models such as decision trees.

• Models can often be trained in parallel.

• Each bootstrap sample contains some repeated rows and leaves out some training rows.

Out-of-bag (OOB) evaluation

For a given bootstrap-trained model, some original training examples are not selected. These are its out-of-bag examples. Predictions for each training example can be aggregated from models for which it was out-of-bag, giving an internal estimate of performance without a separate validation split.

Limitation: bagging may not substantially reduce bias, and the ensemble is often less interpretable than a single tree.

5. Random Forest

Definition: Random Forest is an ensemble of decision trees that combines bagging with random feature selection. Each tree is trained on a bootstrap sample, and at each split only a random subset of features is considered.

Random Forest algorithm

1. Create a bootstrap sample for each tree.
2. Grow a decision tree using a random subset of candidate features at each split.
3. Repeat to build many trees.
4. Combine tree predictions: majority vote for classification, average for regression.

Why random feature selection?

If all trees repeatedly use the same strongest feature, their predictions may be highly correlated. Considering random feature subsets encourages tree diversity, which can make the combined prediction more stable.

Advantages

• Often performs well on tabular datasets.

• Can model nonlinear relationships and interactions.

• Usually needs little feature scaling.

• Can estimate feature importance and provide OOB performance estimates.

Limitations

• Can require more memory and prediction time than one tree.

• The combined forest is harder to explain than a single decision tree.

• Feature importance can be misleading with correlated features or high-cardinality variables.

• It may be less effective than simpler models on some small or structured problems.

6. Boosting

Definition: Boosting builds an ensemble sequentially. Each new learner is trained to improve the combined model, often by paying more attention to difficult examples or by fitting the remaining errors.

Bagging vs. boosting

Bagging	Boosting
Models are commonly trained independently and in parallel.	Models are added sequentially; later stages depend on earlier results.
Mainly reduces variance.	Can reduce bias and improve fit; may also reduce error through iterative refinement.
Each model gets a bootstrap sample in standard bagging.	Learners are reweighted or trained on residuals/gradients, depending on the method.
Example: Random Forest.	Examples: AdaBoost, Gradient Boosting, XGBoost.

7. AdaBoost

AdaBoost means Adaptive Boosting. In its classic binary-classification form, it trains weak learners in sequence and increases the weights of examples that earlier learners misclassified. Correctly classified examples receive relatively less weight. Each learner's influence in the final prediction depends on its performance.

Conceptual steps

1. Give all training examples equal initial weights.
2. Train a weak learner, often a shallow decision tree (decision stump).
3. Measure its weighted classification error.
4. Increase attention to misclassified examples and reduce attention to correctly classified examples.
5. Train the next learner and combine learner predictions using weighted voting.

Advantages: can turn simple learners into a strong classifier. **Limitations:** classic AdaBoost can be sensitive to noisy labels and outliers because difficult examples may receive high weights.

8. Gradient Boosting

Gradient Boosting builds an additive model in stages. Each new learner, commonly a shallow decision tree, is fitted to improve the current model by following the gradient of a chosen loss function. For squared-error regression, this can be understood as fitting the residual errors.

Conceptual example

Suppose the true target is 100 and the current model predicts 70. The residual is 100 – 70 = 30. A later learner attempts to model this remaining error; its contribution is scaled and added to the existing prediction. In real gradient boosting, the process is repeated across many examples and uses a specified loss function.

Learning rate

The learning rate controls how strongly each new learner contributes. A smaller learning rate often requires more trees but can improve generalization when tuned appropriately.

Important parameters

• **Number of estimators:** number of boosting stages or trees.

• **Learning rate:** contribution of each stage.

• **Tree depth:** complexity of each base learner.

• **Subsample:** fraction of training rows used per stage in stochastic variants.

9. XGBoost

XGBoost (Extreme Gradient Boosting) is an optimized implementation of gradient-boosted decision trees. It adds engineering and algorithmic improvements such as regularization, efficient split finding, and support for missing-value handling.

Why it is popular

• Often gives strong performance on structured or tabular data.

• Includes regularization to help control model complexity.

• Supports learning-rate, tree-depth, sampling, and other tuning controls.

• Offers efficient implementations for many practical datasets.

XGBoost is not automatically the best model for every dataset. It should be compared with a suitable baseline and tuned using validation data.

10. Stacking

Definition: Stacking (stacked generalization) combines predictions from multiple base models using another model called a meta-learner.

How stacking works

1. Train different base models, such as a decision tree, logistic regression, and support-vector machine.
2. Generate out-of-fold predictions from the base models on the training data.
3. Use those predictions as inputs to train the meta-learner.
4. At prediction time, obtain base-model predictions and pass them to the trained meta-learner.

Critical precaution: use out-of-fold predictions to train the meta-model. Training the meta-model on predictions made on the same rows used to fit the base models can cause leakage and overly optimistic results.

11. Choosing an Ensemble Method

Situation	Possible starting point	Reason / caution
Need an interpretable baseline	Single decision tree or linear model	A large ensemble may be harder to explain.
Want stable tree predictions	Random Forest	A strong general-purpose tabular baseline.
Need high performance on tabular data	Gradient Boosting or XGBoost	Tune learning rate, depth, and number of trees.
Have diverse models with complementary errors	Voting or Stacking	Validate carefully; combining weak or similar models may not help.

12. Evaluation and Good Practice

• Choose a metric appropriate to the task: accuracy, precision, recall, F1-score, ROC-AUC for classification; MAE, RMSE, or R^2 for regression.

• Use a training/validation split or cross-validation for model and hyperparameter selection.

• Keep the test set separate until final evaluation.

• Check class imbalance; accuracy alone may be misleading.

• Compare the ensemble with a simple baseline to confirm that added complexity is useful.

• For time-series data, use time-aware splits rather than random shuffling when that would leak future information.

13. Advantages and Limitations of Ensemble Learning

Advantages	Limitations
Can improve accuracy and robustness.	More computation, memory, and tuning may be required.
Can reduce variance or improve model fit, depending on the method.	Often less interpretable than one simple model.
Can combine diverse learners and capture complex patterns.	May overfit if poorly tuned or evaluated with leakage.

Advantages	Limitations
Useful for classification and regression.	No ensemble is guaranteed to beat a well-chosen single model.

14. Short Exam Questions and Answers

1. Define ensemble learning.

Ensemble learning combines predictions from multiple models to produce a final prediction that may be more accurate or robust.

2. What is bagging?

Bagging trains models on different bootstrap samples and combines their predictions.

3. What is a bootstrap sample?

A sample created by drawing training examples with replacement.

4. Define Random Forest.

Random Forest is an ensemble of decision trees using bootstrap samples and random feature subsets at each split.

5. What is boosting?

Boosting builds models sequentially so later learners improve the current ensemble.

6. Differentiate bagging and boosting.

Bagging generally trains independent models in parallel; boosting trains dependent models sequentially.

7. What is AdaBoost?

An adaptive boosting method that changes example weights so later learners focus more on earlier mistakes.

8. What is Gradient Boosting?

A sequential method that adds learners to reduce a chosen loss, often by fitting residuals or loss gradients.

9. What is XGBoost?

An optimized, regularized implementation of gradient-boosted decision trees.

10. Define stacking.

Stacking trains a meta-learner to combine predictions from multiple base models.

11. What is hard voting?

Selecting the class with the most votes from the classifiers.

12. What is soft voting?

Combining predicted class probabilities and selecting the class with the highest combined probability.

13. What is out-of-bag evaluation?

An internal performance estimate using training examples omitted from particular bootstrap samples.

14. Why is a learning rate used in boosting?

It controls the contribution of each new learner to the combined model.

15. State two limitations of ensemble learning.

It may require more computation and may be harder to interpret than a single model.

15. Long-Answer Practice

• Define ensemble learning and explain its benefits and limitations.

• Explain bagging and Random Forest with step-by-step algorithms.

• Compare bagging, boosting, and stacking with examples.

• Describe AdaBoost and explain how it changes the weights of training examples.

• Explain Gradient Boosting, the role of residuals, and the learning rate.

• What is XGBoost? Discuss its key features and common tuning parameters.

• Explain hard voting, soft voting, and averaging with examples.

• Discuss the evaluation precautions required to avoid data leakage in ensemble learning.

Quick Revision

Term	Remember
Ensemble	Combines multiple model predictions.
Bagging	Bootstrap samples; parallel models; average or vote.
Random Forest	Bagged trees plus random feature selection.
Boosting	Sequential learners improve the current ensemble.
AdaBoost	Reweights training examples based on errors.
Gradient Boosting	Adds learners to reduce a loss function.
XGBoost	Efficient, regularized gradient-boosted trees.
Stacking	Meta-model combines base-model predictions.

DATA SCIENCE

MODULE 5

DIMENSIONALITY REDUCTION

Study Notes • Concepts • PCA & SVD • Examples • Exam Questions

Learning objectives

• Understand the curse of dimensionality and why dimensionality reduction is useful.

• Differentiate feature selection from feature extraction.

• Explain Principal Component Analysis (PCA) and Singular Value Decomposition (SVD).

• Understand variance, covariance, eigenvectors, and eigenvalues in PCA.

• Learn other reduction techniques and practical implementation precautions.

Beginner-friendly classroom notes and examination revision material.

1. Introduction to Dimensionality

Definition: Dimensionality refers to the number of features or variables used to describe each data observation. A dataset with 100 input columns has 100 dimensions in its feature space.

Dimensionality reduction is the process of reducing the number of input dimensions while trying to preserve the most useful information in the data.

Why reduce dimensionality?

• **Reduce computation:** fewer variables can make training and prediction faster.

• **Reduce noise and redundancy:** some features may repeat information or contribute little useful signal.

• **Help visualization:** high-dimensional data can be projected to two or three dimensions for plotting.

• **Reduce overfitting risk:** fewer effective degrees of freedom may help models generalize.

• **Improve storage and efficiency:** compact representations may require less memory.

The curse of dimensionality

As the number of dimensions grows, data points become sparse in the feature space. Distance and neighborhood measures may become less informative, and much more data can be needed to learn reliable patterns. This collection of difficulties is called the curse of dimensionality.

Example: a model trained with 3 useful features may be easier to fit with limited data than one using hundreds of noisy features, although the best choice depends on the task.

2. Feature Selection vs. Feature Extraction

Feature Selection	Feature Extraction
Keeps a subset of original features.	Transforms original features into a smaller set of new features.
Original meaning is often easier to explain.	New components may be combinations of many original features.
Example: keep Age, Income, and Score from 30 columns.	Example: PCA transforms 30 columns into 5 principal components.

Feature selection chooses which columns to keep; dimensionality-reduction methods such as PCA often create new coordinates from the original columns.

3. Principal Component Analysis (PCA)

Definition: PCA is an unsupervised linear dimensionality-reduction technique that transforms correlated numeric variables into a smaller number of new, mutually orthogonal variables called principal components. The first component captures the greatest possible variance; each following component captures the greatest remaining variance subject to being orthogonal to earlier components.

Key terms

• **Variance:** measures how spread out values are.

• **Covariance:** indicates how two variables vary together.

• **Principal component:** a weighted linear combination of the original features.

• **Eigenvector:** a direction associated with a matrix transformation; in PCA it gives a principal direction.

• **Eigenvalue:** the amount of variance associated with a principal direction.

Steps of PCA

1. Arrange data as rows = observations and columns = features.
2. Clean missing values and usually standardize features when they have different units or scales.
3. Subtract each feature's mean (center the data).
4. Compute the covariance matrix, or use an equivalent decomposition of the centered data matrix.
5. Find eigenvalues and eigenvectors of the covariance matrix.
6. Sort components by decreasing eigenvalue.
7. Choose the first k components based on the desired information retention.
8. Project the centered data onto those k component directions.

Projection: if X is the centered data matrix and W contains the selected component directions, the reduced representation is Z = XW.

Why standardize features?

Suppose a dataset contains Age measured in years and Salary measured in thousands of rupees. Salary may have much larger numerical variance simply because of its scale. Standardization places features on comparable scales, often using z = (x – mean) / standard deviation. Whether to standardize depends on the data and task; it is generally important when units differ substantially.

Explained variance ratio

For component i, the explained variance ratio is its eigenvalue divided by the sum of all eigenvalues. The cumulative explained variance ratio adds these ratios from the first component through component k.

Example: if the first three components explain 50%, 30%, and 10% of the variance, retaining two components preserves 80% of the variance, while retaining three preserves 90%.

Advantages and limitations of PCA

• **Advantages:** reduces correlated dimensions, can compress data, may reduce noise, and supports 2D/3D visualization.

• **Limitations:** components may be hard to interpret; PCA is linear; high variance is not always the same as high predictive value; some information is lost when components are discarded.

4. Small Conceptual PCA Example

Imagine a dataset with two highly correlated measurements: height in centimeters and height in meters. They carry nearly the same information. PCA can find a direction that captures their shared variation and a second direction with very little variance. Keeping only the first component can represent most of the information with one dimension.

This example is conceptual: the actual PCA result depends on centering, scaling, and the values in the dataset.

5. Singular Value Decomposition (SVD)

Definition: Singular Value Decomposition factors a matrix A into three matrices: A = UΣV■. It is a general matrix factorization used in dimensionality reduction, least-squares problems, compression, and other numerical methods.

Meaning of the matrices

• **U:** contains left singular vectors.

• Σ **(Sigma):** diagonal matrix containing non-negative singular values, typically ordered from largest to smallest.

• **V■:** transpose of the matrix of right singular vectors.

A truncated SVD keeps only the largest k singular values and corresponding vectors to form a lower-rank approximation of the original matrix. This can retain dominant patterns while discarding smaller components.

Relationship to PCA: applying SVD to a centered data matrix provides principal directions and singular values from which PCA components and explained variance can be derived. SVD is often used to compute PCA numerically without explicitly forming the covariance matrix.

PCA vs. SVD

PCA	SVD
A statistical method that identifies directions of maximum variance.	A general factorization of a matrix.
Often explained using covariance, eigenvectors, and eigenvalues.	Expresses a matrix as UΣV■.
Usually centers the data; scaling depends on the task.	Can be applied to many matrices; for PCA, centering is normally required.

6. Other Dimensionality-Reduction Techniques

A. Linear Discriminant Analysis (LDA)

LDA is a supervised technique that finds projections intended to separate known classes. Unlike PCA, which does not use class labels, LDA uses labels and aims to increase separation between classes relative to variation within each class. In its standard form, the number of components is limited by the number of classes minus one.

B. t-SNE

t-distributed Stochastic Neighbor Embedding (t-SNE) is a nonlinear technique mainly used to visualize high-dimensional data in two or three dimensions. It aims to preserve local neighborhood relationships. Distances between separate clusters and cluster sizes in a t-SNE plot should not automatically be interpreted as global structure.

C. UMAP

Uniform Manifold Approximation and Projection (UMAP) is another nonlinear technique often used for visualization and exploratory analysis. It can preserve local structure and sometimes broader relationships, but results depend on settings and should be interpreted cautiously.

D. Autoencoders

An autoencoder is a neural network trained to reconstruct its input through a smaller internal representation called a latent code. The encoder maps input data to the compact code, and the decoder attempts to reconstruct the original input.

Technique	Supervision	Common purpose
PCA	Unsupervised	Linear compression and variance preservation.
LDA	Supervised	Project data to help separate labeled classes.
t-SNE	Usually unsupervised	Low-dimensional visualization of local neighborhoods.
UMAP	Usually unsupervised	Nonlinear embedding and visualization.
Autoencoder	Usually self-supervised reconstruction	Learn compact nonlinear representations.

7. Choosing the Number of Components

There is no single correct number of dimensions for every dataset. Common approaches include:

• **Explained variance threshold:** retain enough PCA components to preserve a chosen percentage, such as 90% or 95%, when appropriate.

• **Scree plot:** plot eigenvalues or explained variance and look for an elbow where additional components add less information.

• **Validation performance:** compare downstream model performance using different numbers of components.

• **Interpretability and cost:** balance compactness against information loss and computational requirements.

A variance threshold is a heuristic, not a guarantee of predictive performance. Components with low variance can sometimes contain information useful for predicting a target.

8. Practical Workflow and Common Mistakes

A safe general workflow is:

1. Split the data into training, validation, and test sets as appropriate.
2. Fit preprocessing steps, including imputation and scaling, using training data only.
3. Fit PCA or another reduction method on training data.
4. Transform validation and test data using the already-fitted preprocessing and reduction steps.
5. Select settings using validation or cross-validation.
6. Evaluate the final pipeline on held-out test data.

Common mistakes

• Fitting PCA on the entire dataset before splitting, which leaks information from test data.

• Forgetting to center data before standard PCA.

• Using PCA without considering scale differences among features.

• Assuming PCA uses target labels; standard PCA is unsupervised.

• Assuming more explained variance always means better classification or regression performance.

• Interpreting t-SNE or UMAP plots as exact maps of global distances.

9. Advantages and Limitations

Advantages	Limitations
Can reduce memory and computation.	Discarded dimensions may contain useful information.
Can remove redundancy and some noise.	Transformed features may be difficult to interpret.
Can make high-dimensional data easier to visualize.	Results depend on scaling, method, and parameters.
Can help some models generalize.	May not improve every model or dataset.

10. Short Exam Questions and Answers

1. Define dimensionality reduction.

Dimensionality reduction reduces the number of features or transforms data into fewer dimensions while preserving useful information.

2. What is the curse of dimensionality?

It describes problems caused by high-dimensional spaces, such as sparse data, less meaningful distances, and greater data requirements.

3. Differentiate feature selection and feature extraction.

Feature selection keeps some original features; feature extraction creates new transformed features.

4. Define PCA.

PCA is an unsupervised linear technique that projects data onto orthogonal directions of maximum variance.

5. What is a principal component?

A principal component is a linear combination of original features representing a direction in the transformed space.

6. What are eigenvalues in PCA?

They indicate the variance associated with the corresponding principal components.

7. Why is standardization sometimes necessary before PCA?

It prevents features with large numerical scales from dominating when their units or variances differ substantially.

8. What is explained variance ratio?

It is the fraction of total variance represented by a principal component.

9. Define SVD.

SVD factors a matrix A into UΣV■, where U and V contain singular vectors and Σ contains singular values.

10. State one relationship between PCA and SVD.

SVD of a centered data matrix can be used to compute PCA directions and explained variance.

11. Differentiate PCA and LDA.

PCA is unsupervised and maximizes variance; LDA uses class labels and seeks directions that separate classes.

12. What is t-SNE used for?

It is mainly used to visualize high-dimensional data in two or three dimensions while emphasizing local neighborhoods.

13. What is an autoencoder?

A neural network that learns a compact latent representation and reconstructs the input.

14. What is a scree plot?

A plot of component eigenvalues or explained variance used to help choose the number of components.

15. What is data leakage in dimensionality reduction?

It occurs when the reduction method is fitted using validation or test data before evaluation.

11. Long-Answer Practice

• Define dimensionality reduction and explain the curse of dimensionality.

• Explain PCA step by step, including centering, covariance, eigenvectors, eigenvalues, and projection.

• Explain explained variance ratio and discuss how to choose the number of principal components.

• Define SVD and explain the meaning of U, Sigma, and V■. Describe its relationship to PCA.

• Compare PCA, LDA, t-SNE, UMAP, and autoencoders.

• Discuss the advantages, limitations, and practical applications of dimensionality reduction.

• Describe a leakage-safe workflow for applying PCA in a machine-learning project.

Quick Revision

Term	Remember
Dimensionality reduction	Represent data using fewer dimensions.
Feature selection	Keep a subset of original columns.
PCA	Unsupervised linear projection; preserves directions of high variance.
Eigenvalue	Variance associated with a principal direction.
Explained variance	Fraction of total variance represented by selected components.
SVD	Matrix factorization A = UΣV■.
LDA	Supervised projection for class separation.
t-SNE / UMAP	Nonlinear embeddings commonly used for visualization.